

THIẾT KẾ NFA

NFA (Nondeterministic Finite Automaton - Máy tự động hữu hạn không đơn định) là một kỹ năng quan trọng trong lý thuyết tính toán. Khác với DFA (máy đơn định) vốn cứng nhắc, NFA cho phép chúng ta "đoán" hoặc đi theo nhiều hướng cùng một lúc.

NFA là gì? (Nhắc lại)

Một NFA được định nghĩa bằng bộ năm $M = (Q, \Sigma, \delta, q_0, F)$.

Trong đó:

Q : tập hữu hạn các trạng thái nội.

Σ : tập hữu hạn các ký hiệu được gọi là bảng chữ cái đầu nhập.

$\delta: Q \times (\Sigma \cup \lambda) \rightarrow 2^Q$ gọi là hàm chuyển dịch.

$q_0 \in Q$ là trạng thái ban đầu.

$F \subseteq Q$ là tập trạng thái kết thúc

Nhắc lại:

Nếu một tập hợp có n phần tử, thì số tập con của nó sẽ là: 2^n

Ví dụ minh họa

Giả sử có tập hợp $A = \{1, 2, 3\}$ gồm 3 phần tử ($n = 3$) thì số tập con là $2^3 = 8$

Tập rỗng: $\{\}$ luôn là tập con của mọi tập hợp

Tập có 1 phần tử: $\{1\}, \{2\}, \{3\}$

Tập có 2 phần tử: $\{1, 2\}, \{1, 3\}, \{2, 3\}$

Tập có 3 phần tử (chính nó): $\{1, 2, 3\}$

Tổng cộng chúng ta có đúng 8 tập con.

$2^A = \{ \{\}, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\} \}$ có 8 phần tử

Sự khác nhau giữa NFA và DFA

Mỗi trạng thái của DFA có số mũi tên đi ra các trạng thái khác bằng số phần tử trong Σ .

Mỗi trạng thái của NFA có mũi tên đi ra và đến một tập các trạng thái. Tập này có thể rỗng hoặc một hay nhiều phần tử. Đặc biệt, từ một trạng thái có thể dịch chuyển (λ) sang một trạng thái khác mà không cần đọc một ký tự nào.

THIẾT KẾ NFA

Thiết kế NFA thì khác với DFA (Đơn định), NFA cho phép chúng ta "ĐOÁN" hoặc đi theo nhiều hướng cùng lúc, giúp việc thiết kế các ngôn ngữ phức tạp trở nên trực quan và ngắn gọn hơn rất nhiều.

Dưới đây là cách thiết kế NFA và các ví dụ minh họa theo từng bước.

1. Tư duy thiết kế NFA

Để thiết kế một NFA hiệu quả, cần nắm vững 3 "điểm chính" mà DFA không có:

1. **Chuyển dịch λ :** Có thể nhảy từ trạng thái này sang trạng thái khác mà không cần đọc ký tự nào.
2. **Nhiều lựa chọn:** Từ một trạng thái, với cùng một ký tự nhập vào, có thể đi đến nhiều trạng thái khác nhau.
3. **Trạng thái chết:** Nếu đọc một ký tự mà không có đường đi tiếp theo, nhánh đó sẽ tự động "chết", không cần phải vẽ trạng thái "bẫy" (trap state) như DFA.

Quy trình thiết kế:

Bước 1: Xác định "MÃU" cốt lõi mà ngôn ngữ yêu cầu.

Bước 2: Tạo trạng thái bắt đầu q_0 . Sử dụng các trạng thái để đại diện cho các giai đoạn của quá trình so khớp mẫu.

Bước 3: Sử dụng tính không đơn định để xử lý các phần "không quan trọng" ở đầu hoặc cuối chuỗi.

Thứ tự các bước thì tùy theo từng đầu bài yêu cầu thiết kế, bởi vì không có giải thuật để thiết kế NFA một cách tổng quát.

2. Các ví dụ thiết kế

Ví dụ 1: Thiết kế NFA cho ngôn ngữ chứa chuỗi con "011"

$\Sigma = \{1, 0\}$ dùng kí hiệu ba dấu *, "***" để mô tả chuỗi gồm các kí tự trong Σ và có độ dài tùy ý có kể đến kí tự rỗng λ

Phân tích:

+ Chúng ta chỉ cần tập trung vào việc tìm thấy chuỗi 011 (MÃU). Trước và sau chuỗi này là chuỗi có thể chứa bất cứ kí hiệu nào.

+ Liệt kê một vài chuỗi của ngôn ngữ mà NFA chấp nhận, trong đó những chuỗi đặc biệt, chuỗi đặc biệt trong ví dụ này là chuỗi chỉ có 3 ký tự là 011.

+ ví dụ có 4 chuỗi hợp lệ: ***011*** ; 011; 001***; ***001

Thiết kế

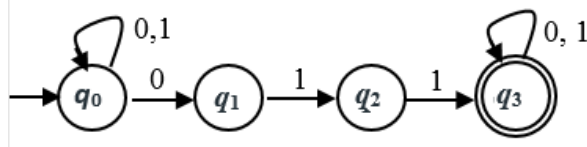
Trạng thái q_0 : Trạng thái bắt đầu. Tại đây, ta có một vòng lặp có nhãn là 0,1 để "ĐỢI" có nghĩa là cho đọc chuỗi *** cho đến khi chuỗi MÃU 011 xuất hiện.

Bước đoán: Tại q_0 , khi đọc 0, NFA có quyền "đoán" rằng đây là khởi đầu của chuỗi 011 và chuyển trạng thái là: $(q_0, 0) = q_1$.

Khi đã đọc được 0 rồi, thì hy vọng đọc tiếp 11 để có MÃU 011 thì phải có những chuyển dịch:

$$(q_1, 1) = q_2 \text{ và } (q_2, 1) = q_3.$$

Trạng thái q_3 : Là trạng thái chấp nhận vì đã đọc được MÃU 001. Và từ đây nếu còn thì đọc chuỗi *** nên tạo vòng lặp có nhãn 0, 1 tại q_3 .



Ví dụ 2: Thiết kế NFA cho ngôn ngữ kết thúc bằng "ab" hoặc "ba"

$\Sigma = \{a, b\}$

Phân tích:

+ Chúng ta chỉ cần tập trung vào việc tìm thấy chuỗi ab hay ba (MÃU) ở cuối chuỗi. Trước chuỗi này là chuỗi có thể chứa bất cứ kí hiệu nào.

+ Liệt kê một vài chuỗi của ngôn ngữ mà NFA chấp nhận, trong đó những chuỗi đặc biệt, chuỗi đặc biệt trong ví dụ này là chuỗi chỉ có 2 ký tự là ab hoặc ba.

+ ví dụ những chuỗi hợp lệ: ***ab ; ab; ba; ***ba

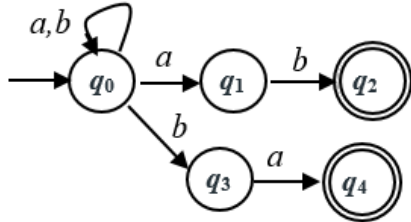
NFA sẽ rẽ nhánh ngay từ đầu.

Nhánh 1: Đoán các chuỗi kết thúc bằng ab.

Nhánh 2: Đoán lấy các chuỗi kết thúc bằng ba.

Thiết kế:

1. q_0 là trạng thái bắt đầu với vòng lặp tại q_0 với nhãn a, b .
2. $(q_0, a) = q_1$, đây là nhánh đoán kết thúc là ab .
3. $(q_0, b) = q_3$, đây là nhánh đoán kết thúc là ba .
4. $(q_1, b) = q_2$. Chấp nhận.
5. $(q_3, a) = q_4$. Chấp nhận.



Ví dụ 3: NFA cho ngôn ngữ có ký tự thứ 3 tính từ cuối lên là '1'

$\Sigma = \{0, 1\}$

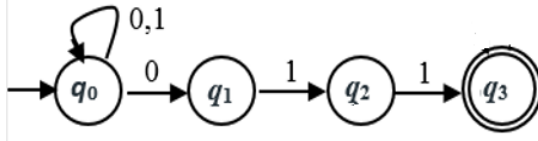
Phân tích:

- + Chúng ta chỉ cần tập trung vào việc tìm thấy chuỗi có ký tự thứ 3 tính từ cuối chuỗi đi ngược lên là ký tự 1. Trước ký tự 1 này có thể chứa bất cứ ký hiệu nào.
- + Liệt kê một vài chuỗi của ngôn ngữ mà NFA chấp nhận,
- + ví dụ những chuỗi hợp lệ: $***1**$ ($**$ là 2 ký tự bất kỳ khác λ)

NFA sẽ "đoán" xem đâu là vị trí thứ 3 tính từ cuối. Khi nó đọc một ký tự 1, nó sẽ thử kích hoạt một luồng kiểm tra xem sau đó có đúng 2 ký tự nữa là kết thúc chuỗi hay không.

Thiết kế:

1. q_0 là trạng thái bắt đầu với vòng lặp tại q_0 với nhãn 0, 1.
2. $(q_0, 1) = q_1$, đoán khi gặp ký tự 1 là ký tự gần cuối, nó đứng trước 2 ký tự ở cuối chuỗi.
3. $(q_1, \{0,1\}) = q_2$: Đọc ký tự sát với ký tự cuối cùng của chuỗi.
4. $(q_2, \{0,1\}) = q_3$: Đọc ký tự cuối cùng của chuỗi, q_3 : Trạng thái chấp nhận.



Ghi chú:

Nếu sau q_3 mà còn ký tự nào nữa chưa đọc, nhánh đó sẽ bị hủy vì q_3 không có đường đi tiếp. Chỉ những nhánh kết thúc đúng tại q_3 mới được chấp nhận.

Ví dụ 4: NFA cho ngôn ngữ chứa chuỗi 01 HOẶC 10

$\Sigma = \{0, 1\}$

Phân tích:

Ngôn ngữ này chấp nhận bất kỳ chuỗi nào có sự xuất hiện của một trong hai mẫu cơ bản (mẫu chứa 01 và mẫu chứa 10).

- + Liệt kê một vài chuỗi của ngôn ngữ mà NFA chấp nhận, trong đó những chuỗi đặc biệt, chuỗi đặc biệt trong ví dụ này là chuỗi chỉ có 2 ký tự là 01 hoặc 10.
- + ví dụ những chuỗi hợp lệ: $***01***$; $***10***$; 01; 10;

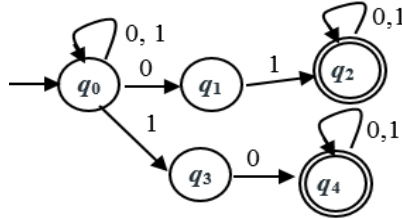
NFA sẽ rẽ nhánh ngay từ đầu.

Nhánh 1: Đoán các chuỗi có chứa chuỗi con 01.

Nhánh 2: Đoán các chuỗi có chứa chuỗi con 10.

Thiết kế:

1. q_0 là trạng thái bắt đầu với vòng lặp tại q_0 với nhãn 0, 1 để đọc các kí tự ở đầu chuỗi.
2. Nhánh 1 để đoán đọc vào 01: $(q_0, 0) = q_1$ và $(q_1, 1) = q_2$, trạng thái q_2 là kết thúc.
3. Nhánh 2 để đoán đọc vào 10: $(q_0, 1) = q_3$ và $(q_3, 0) = q_4$, trạng thái q_4 là kết thúc.
4. Tại q_2 và q_4 có vòng lặp có nhãn là 0,1.



Ví dụ 5: NFA cho ngôn ngữ có ít nhất 2 ký tự và ký tự đầu và ký tự cuối đều giống nhau

$\Sigma = \{a, b\}$

Phân tích:

NFA chấp nhận chuỗi có kí tự đầu và cuối là a hay là b .

Ví dụ: NFA chấp nhận các chuỗi: $a***a$; $b***b$; ...

Thiết kế:

NFA rẽ nhánh từ trạng thái bắt đầu q_0 để tách biệt hai trường hợp: chuỗi bắt đầu bằng a và chuỗi bắt đầu bằng b .

+ Trường hợp 1 (Bắt đầu bằng a):

$(q_0, a) = q_1$, tại q_1 có vòng lặp có nhãn a, b để đọc các ký tự ở giữa sau kí tự a đầu tiên

$(q_1, a) = q_2$, q_2 là trạng thái chấp nhận.

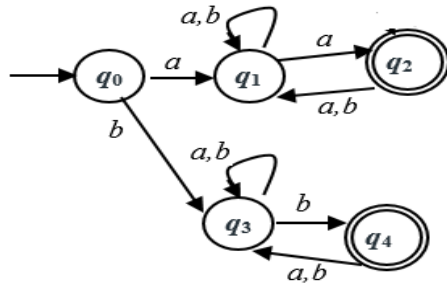
Lưu ý: q_2 cũng có thể quay lại q_1 nếu có ký tự khác được đọc vào, có nghĩa là chuỗi đọc vào vẫn chưa kết thúc.

+ Trường hợp 2 (Bắt đầu bằng b): tương tự trường hợp 1

$(q_0, b) = q_3$, tại q_3 cũng có vòng lặp có nhãn a, b để đọc các ký tự ở giữa sau kí tự b đầu tiên

$(q_3, b) = q_4$, q_4 là trạng thái chấp nhận.

Lưu ý: q_4 cũng có thể quay lại q_3 nếu có ký tự khác được đọc vào, có nghĩa là chuỗi đọc vào vẫn chưa kết thúc.



Ta cũng có thể bỏ đi đường đi từ $q_2 \rightarrow q_1$ và $q_4 \rightarrow q_3$.

Ví dụ 6: NFA chấp nhận các chuỗi có chuỗi con có số lượng ký số 0 là bội số của 2 hoặc chuỗi con có số lượng ký số 1 là bội số của 3 trên $\Sigma = \{0, 1\}$.

Giải

Phân tích

Chuỗi được NFA chấp nhận là chuỗi. Trong đó:

- Chuỗi con chỉ có ký số 0 và số ký số 0 là số chẵn, ví dụ: chuỗi: 00, 0000, ...

- hoặc chuỗi con chỉ có ký số 1, số kí số ký số 1 là bội số của 3

Ví dụ: những chuỗi được NFA chấp nhận: 00, 0000, 000000,... hoặc 111, 111111, ...

Thiết kế:

Ta chỉ cần thiết kế hai máy con A và B rồi nối chúng lại.

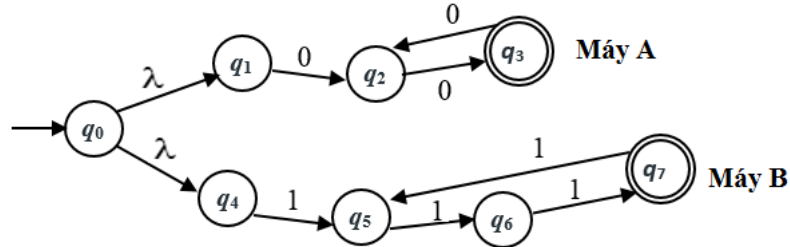
1. **Máy A:** Chấp nhận chuỗi, chứa chuỗi con có số lượng kí số 0 là bội số của 2 (là những số chẵn: 2, 4, 6,...)

$(q_1, 0) = q_2$; $(q_2, 0) = q_3$; $(q_3, 0) = q_2$; q_3 là trạng thái kết thúc.

2. **Máy B:** Chấp nhận chuỗi, chứa chuỗi con có số lượng kí số 1 là bội số của 3.

$(q_4, 1) = q_5$; $(q_5, 1) = q_6$; $(q_6, 1) = q_7$; $(q_7, 1) = q_4$; q_7 là trạng thái kết thúc.

3. **Kết nối:** Tạo một trạng thái bắt đầu q_0 , từ đó vẽ hai cạnh đi ra có nhãn λ dẫn đến trạng thái q_1 của **Máy A** và q_4 của **Máy B**.



Kết quả: Khi một chuỗi đi vào, NFA sẽ "phân nhánh" vào cả hai máy. Chỉ cần một trong hai máy báo "Chấp nhận" ở cuối chuỗi, toàn bộ chuỗi đó sẽ được chấp nhận.

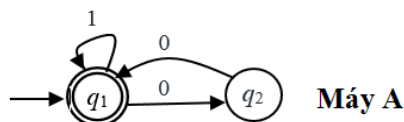
Ví dụ 7: Thiết kế NFA trên $\Sigma = \{0, 1\}$ chấp nhận các chuỗi có chứa những chuỗi con w và v đan xen nhau, w chỉ chứa một hay nhiều ký số 1, v chỉ chứa chẵn kí số 0 không có kí số 1. Ví dụ: 1100, 000010011 là những chuỗi hợp lệ.

HOẶC chấp nhận những chuỗi có chứa những chuỗi con m và n đan xen nhau, m chỉ chứa một hay nhiều ký số 0, n chỉ chứa toàn là kí số 1 sao cho số kí số 1 là bội số của 3.

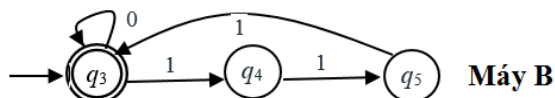
Giải**Phân tích**

- chuỗi v bắt buộc chỉ toàn là ký số 0 và số kí số 0 phải là chẵn
- chuỗi n bắt buộc toàn là kí số 1 và số kí số 1 phải là bội số của 3

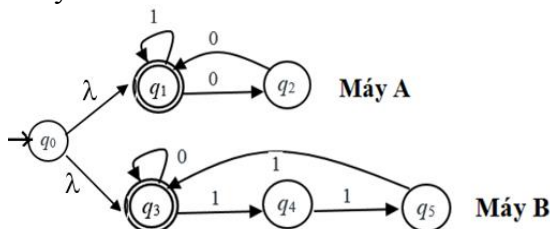
Thiết kế máy A thỏa mãn chuỗi con w và v



Thiết kế máy B thỏa mãn chuỗi con m và n



Kết nối 2 máy A và B với nhãn λ ta có NFA cần thiết kế



BÀI TẬP

- 1- Thiết kế NFA chấp nhận ngôn ngữ L_1 trên bảng chữ cái $\Sigma = \{0, 1\}$ sao cho các chuỗi có ký tự thứ 3 tính từ cuối lên là số 1.
- 2- Thiết kế NFA chấp nhận ngôn ngữ L_2 trên bảng chữ cái $\Sigma = \{a, b\}$ chứa chuỗi con aba hoặc bab .
- 3- Thiết kế NFA chấp nhận các chuỗi nhị phân đại diện cho số chia hết cho 3 hoặc chia hết cho 5.